

Machine Learning Primer



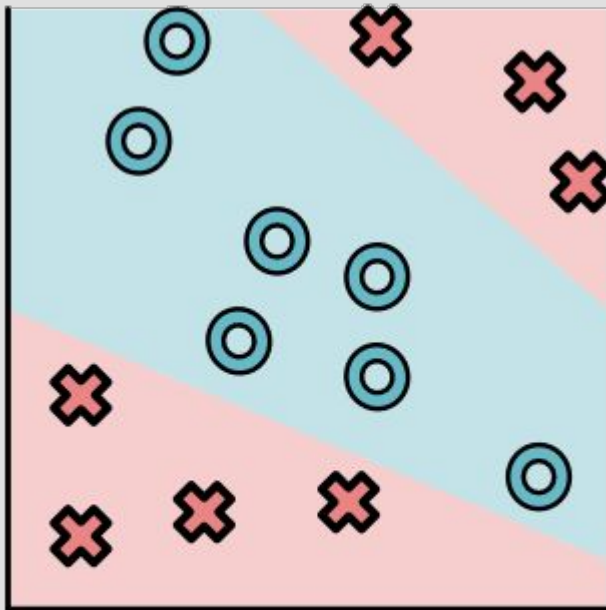
CSCE 5218 / CSCE 4930 Deep Learning
Lecture 1

(content from <https://minitorch.github.io/mlprimer.html>)

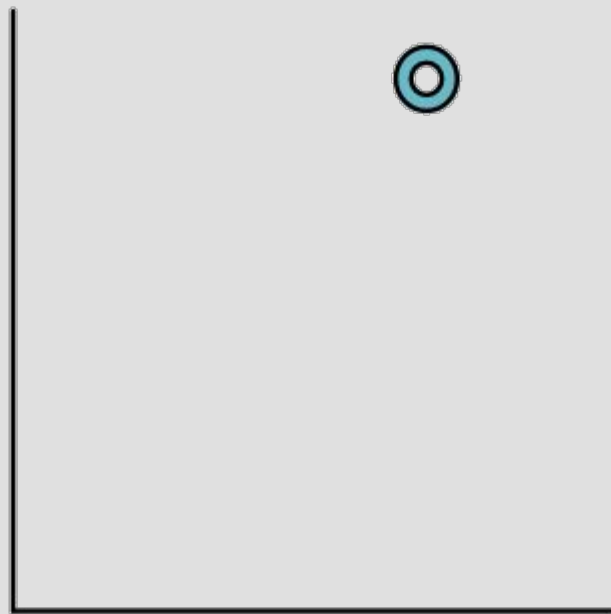
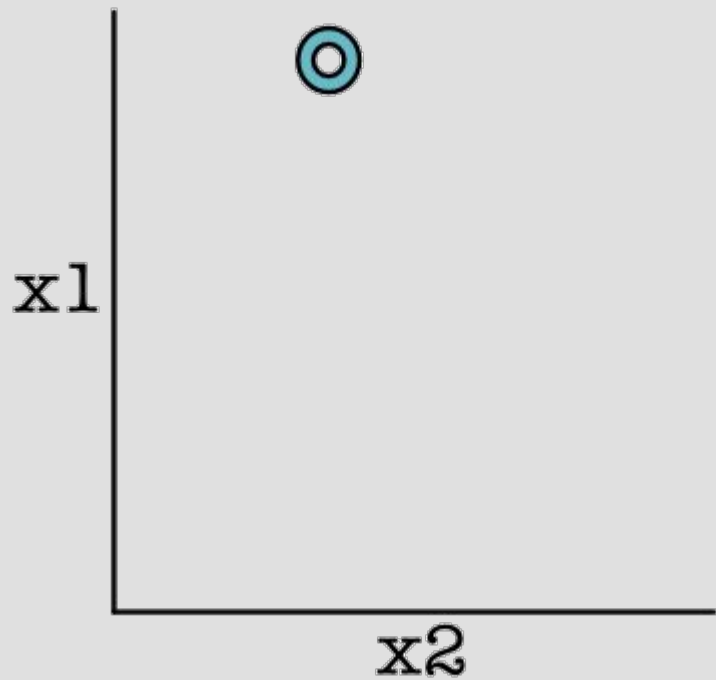
Dataset and Model

- Supervised learning problems begin with a labeled ***training dataset***
- A dataset is a set of points (point == instance == example)
 - a point has some coordinates (aka attributes, features) and a label
 - x : coordinates
 - y : label
 - The problem is to predict y given x
- A *model* is a function to go from coordinates (x) to labels (y)
 - it can be as simple or as complicated as you want to
- We will learn a model from the training dataset
 - and evaluate the model with a separate dataset (the test / evaluation dataset)

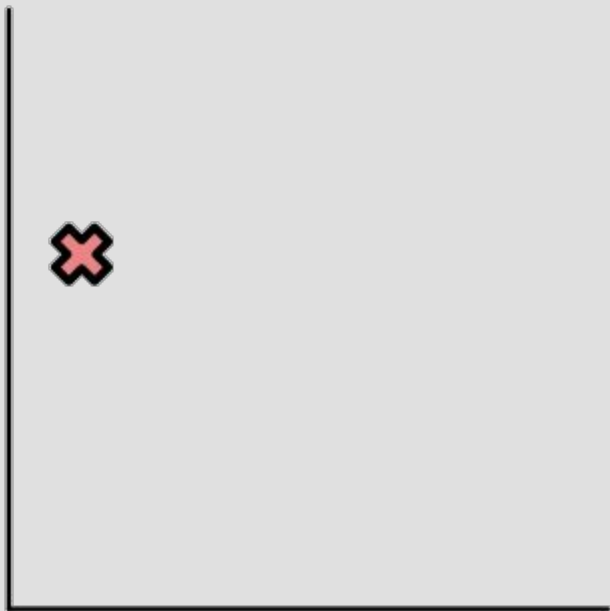
A Brief Guide to Supervised Binary Classification



Datasets

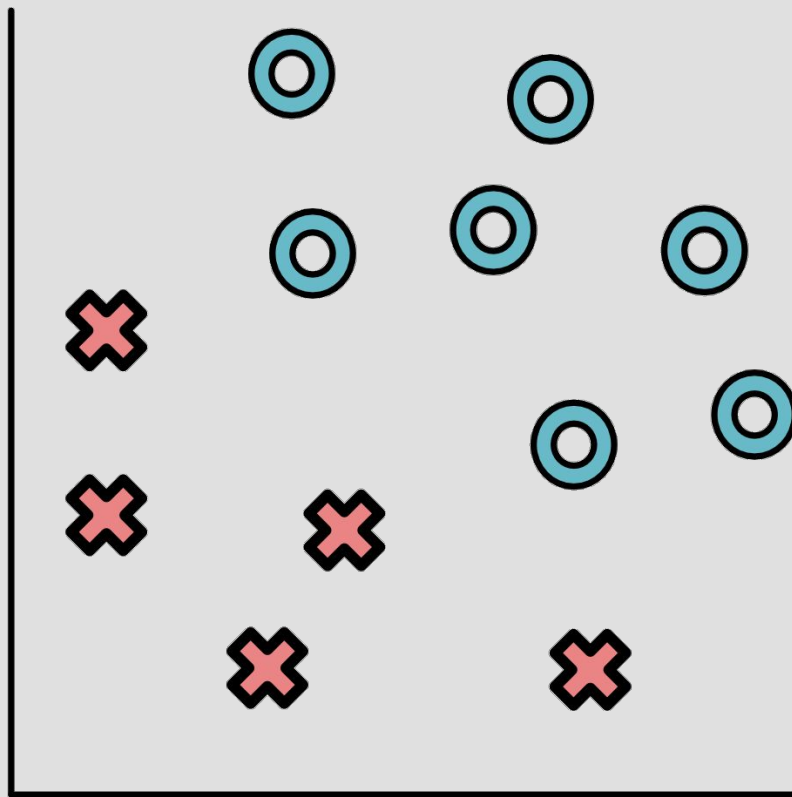


Datasets

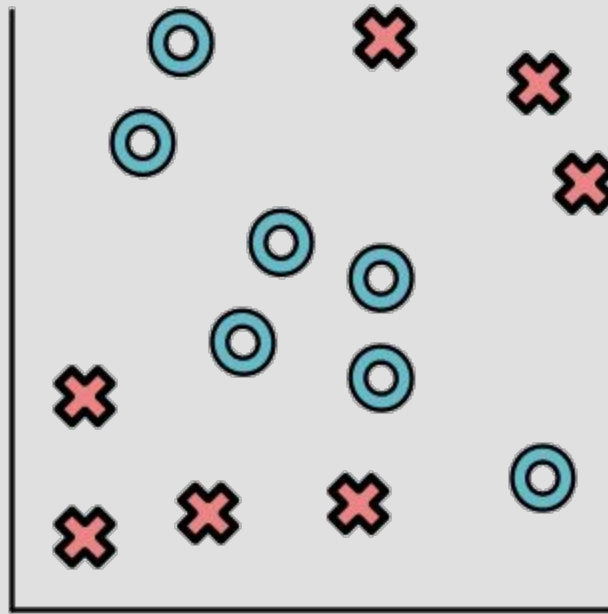


Datasets

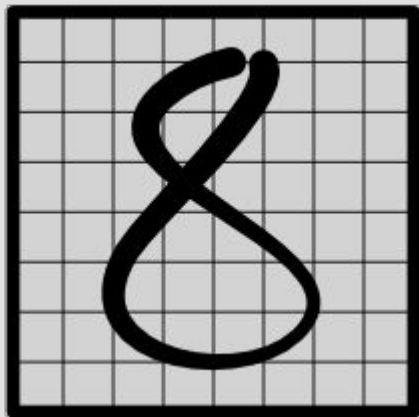
Data



Datasets



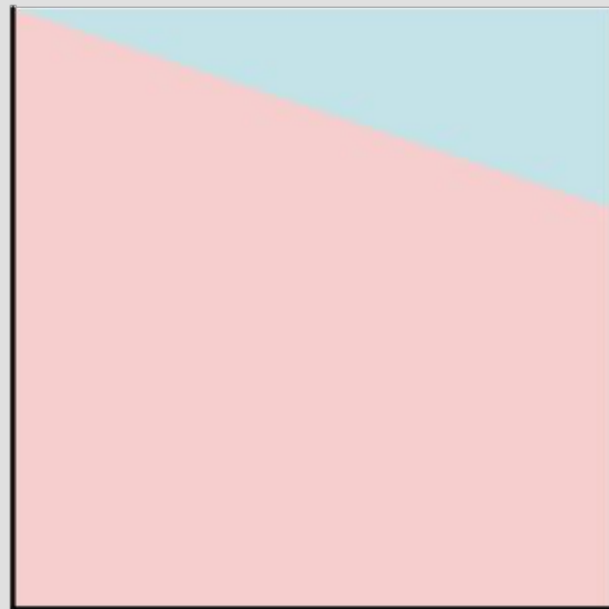
Datasets - More Complicated



Models

A *model* is a function that assigns labels to data points.

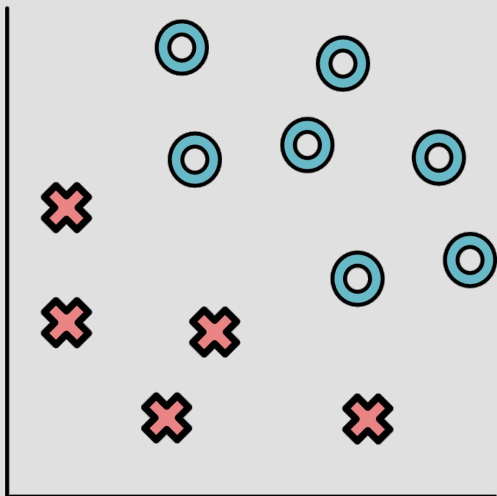
Model



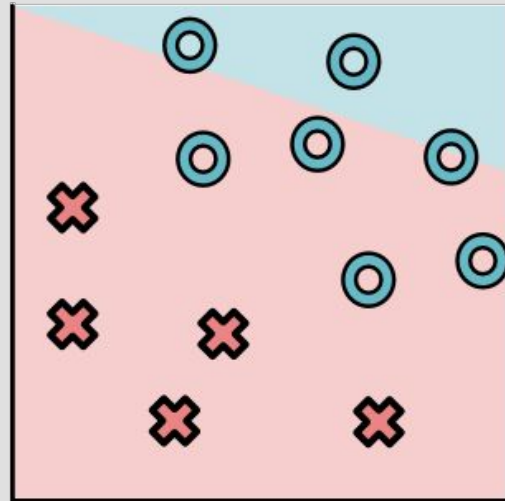
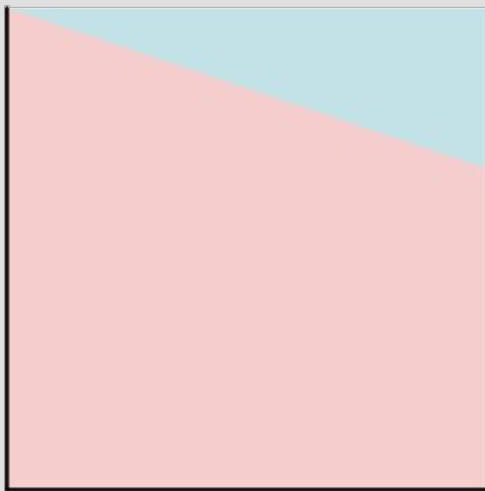
Models

A *model* is a function that assigns labels to data points.

Data

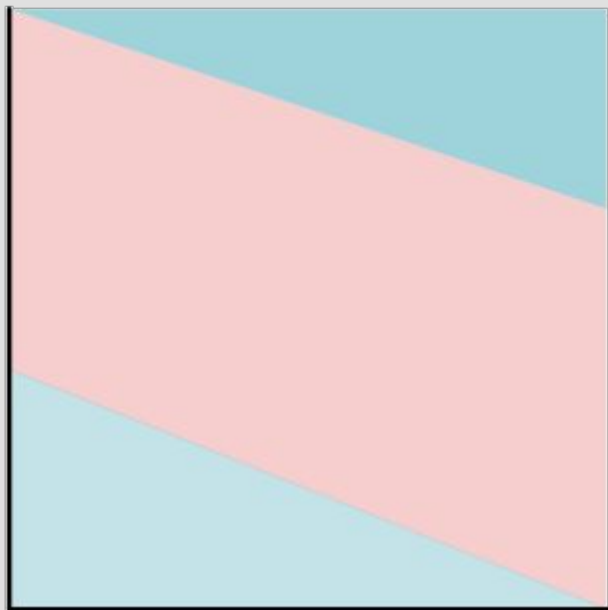


Model

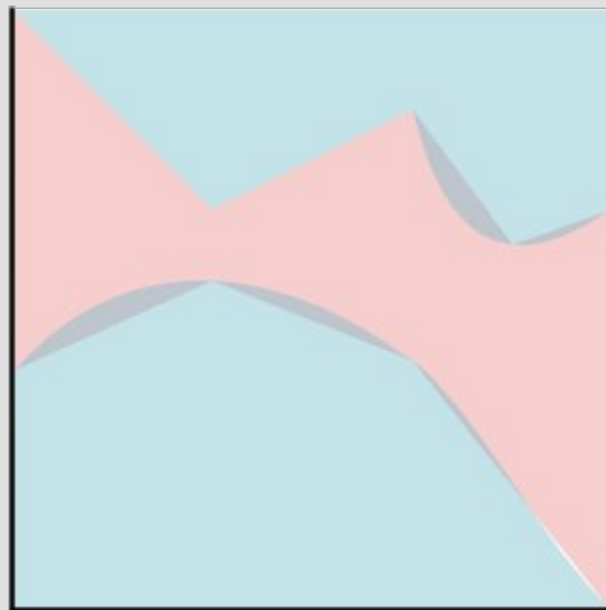


Models

Model

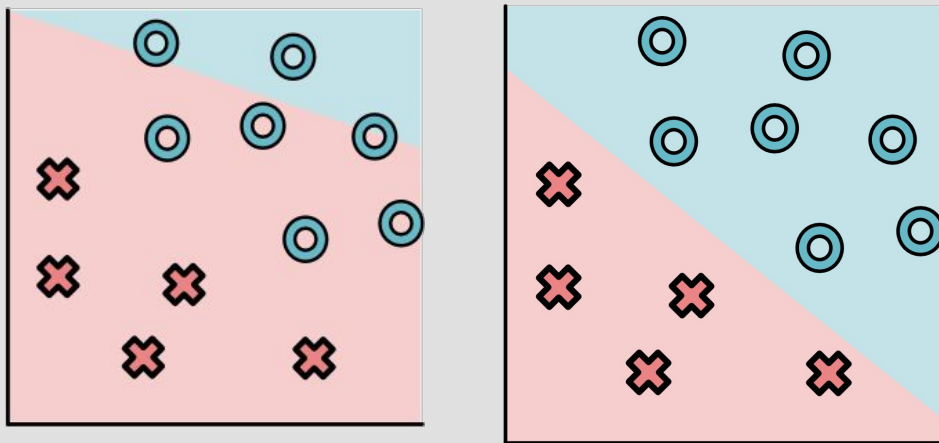


Model



Models

- A *model class* specifies the general shape of models that you want to explore
 - shape == decision boundary
- We are interested to find the best model from a class
- Simple model class: *linear models*
 - decision boundary is a straight line (and only one)



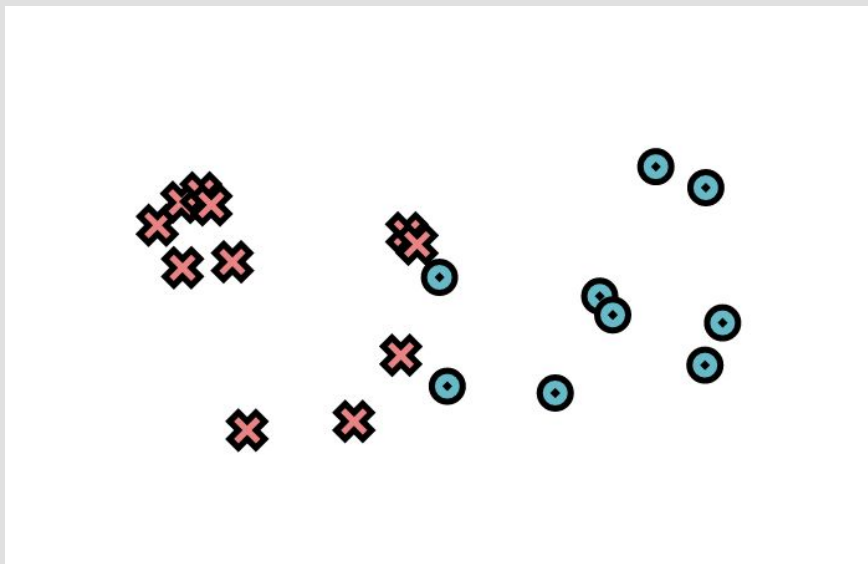
Both models belong to the class of *linear models*

The only difference are the parameters

We will learn parameters automatically (from the training dataset)

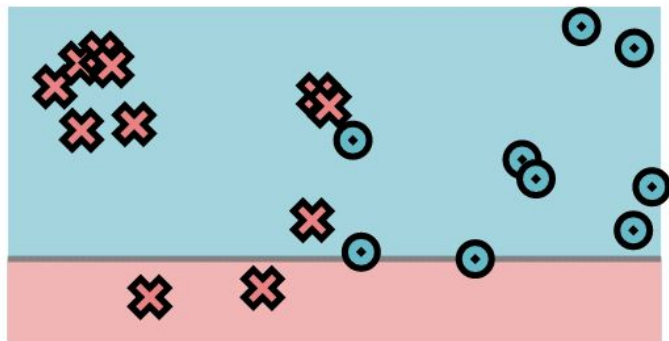
Models

Another dataset:



Models

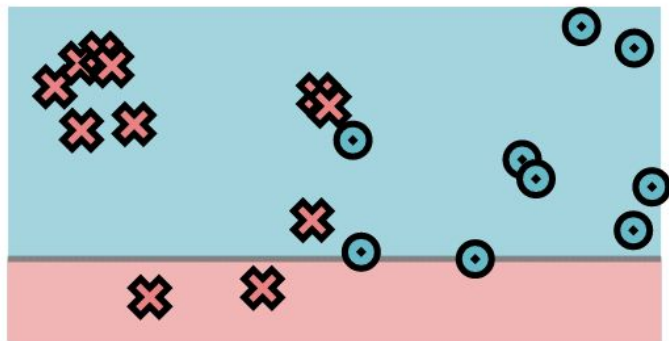
A model for the dataset:



Models

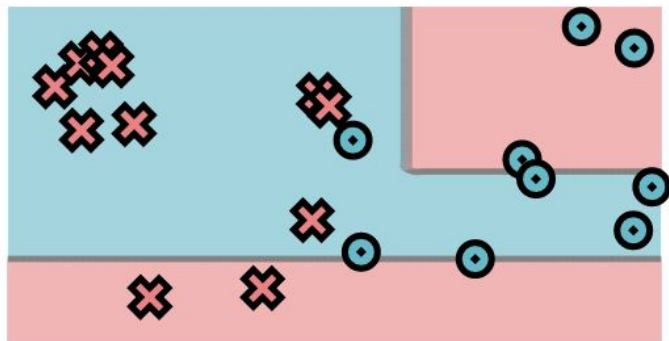
A model for the dataset:

```
def model1(x):  
    if x[0] < 0.25:  
        return 1  
    else:  
        return 0  
plot(s, model1)
```



Models

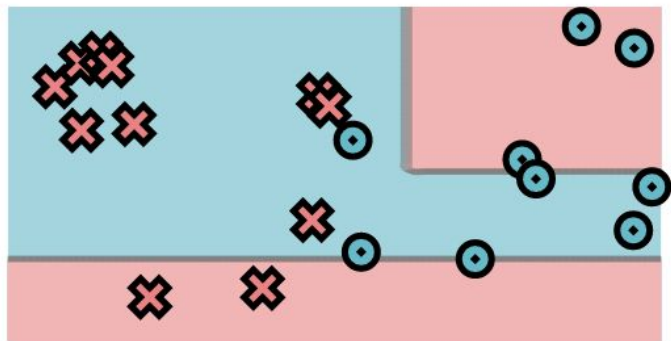
Another model for the dataset:



Models

Another model for the dataset:

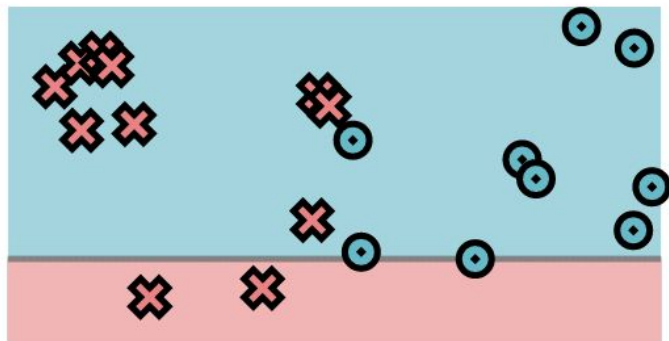
```
def model2(x):  
    if x[0] < 0.25:  
        return 1  
    if x[0] > 0.5 and x[1] >  
0.6:  
        return 1  
    else:  
        return 0  
plot(s, model2)
```



Models

A model for the dataset:

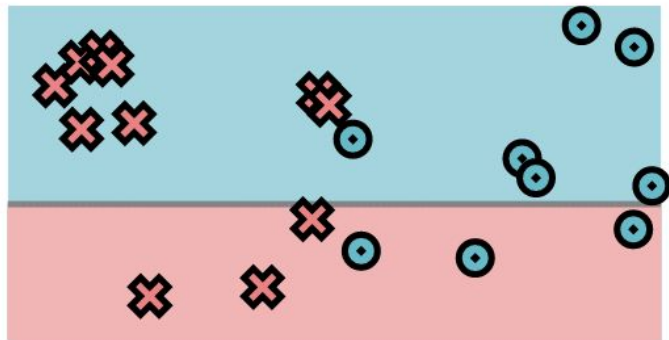
```
def model1(x):  
    if x[0] < 0.25:  
        return 1  
    else:  
        return 0  
plot(s, model1)
```



Models

There are many *linear models*:

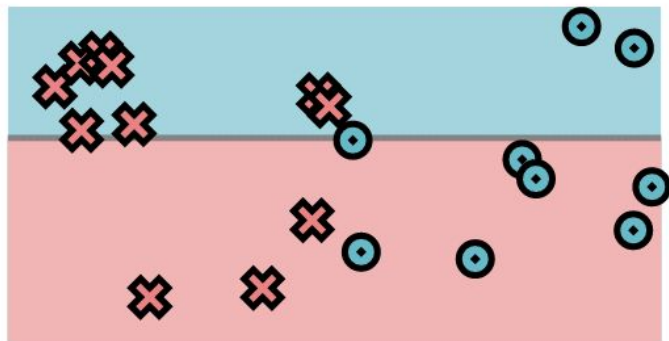
```
def make_model(param):  
    def model(x):  
        if x[0] < param:  
            return 1  
        else:  
            return 0  
    return model  
plot(s, make_model(0.4))
```



Models

There are many *linear models*:

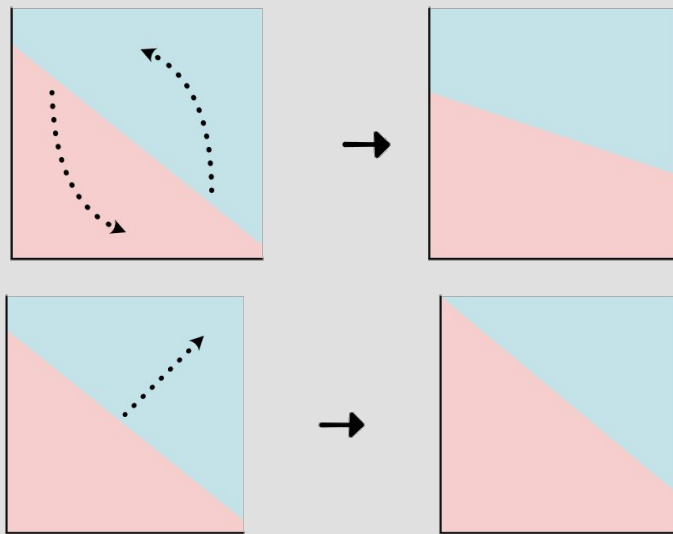
```
plot(s, make_model(0.6))
```



Models

There are many *linear models* depending on the parameters

- You can rotate the boundary
 - aka modify the *slope*
- You can change the separator cutoff
 - aka modify the *intercept*



Models

Linear models for binary classification look like this:

$$m(x_1, x_2; w_1, w_2, b) = x_1 \times w_1 + x_2 \times w_2 + b$$



input (points)

parameters

slope

intercept

```
def make_linear(w1, w2, b):  
    def model(x):  
        return 1 if (x[0] * w1 + x[1] * w2 + b > 0.0) else 0  
    return model
```

Models

Linear models for binary classification look like this:

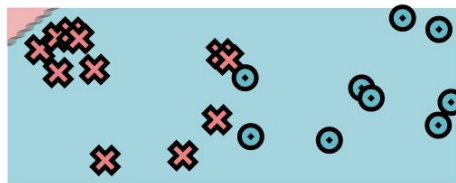
$$m(x_1, x_2; w_1, w_2, b) = x_1 \times w_1 + x_2 \times w_2 + b$$

input (points)

parameters

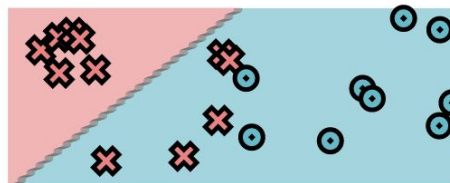
slope

intercept



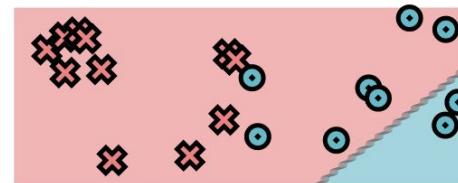
$b = -0.078$

-0.098 -0.068 -0.038 -0.008 0.022 0.052 0.082 0.112 0.142



$b = 0.002$

-0.098 -0.068 -0.038 -0.008 0.022 0.052 0.082 0.112 0.142



$b = 0.132$

-0.098 -0.068 -0.038 -0.008 0.022 0.052 0.082 0.112 0.142

Models

Linear models for binary classification look like this:

$$m(x_1, x_2; w_1, w_2, b) = x_1 \times w_1 + x_2 \times w_2 + b$$

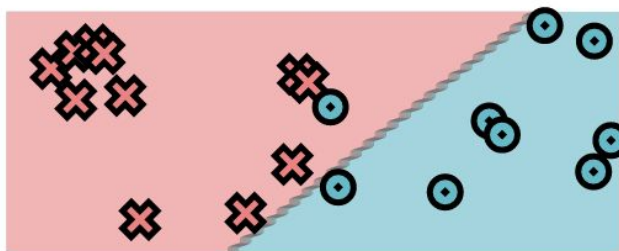


input (points)

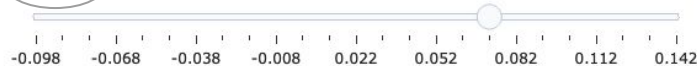
parameters

slope

intercept

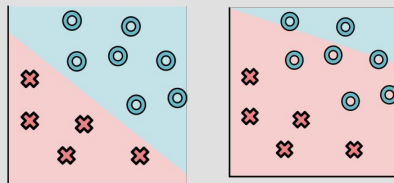


b=0.072

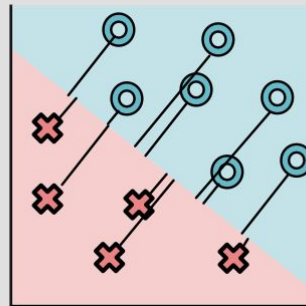


Loss

- Some models are better than others

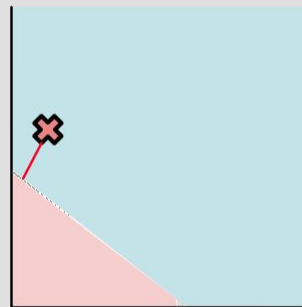
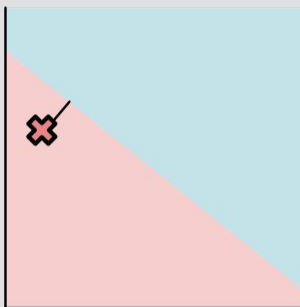
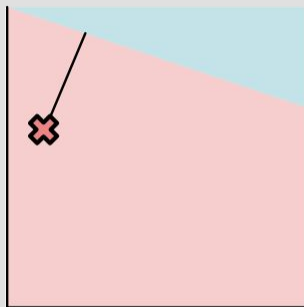


- Loss scores how badly the model is (*not* how good)
 - the smaller the loss, the better
 - ML people like to minimize (*not* maximize)



Loss

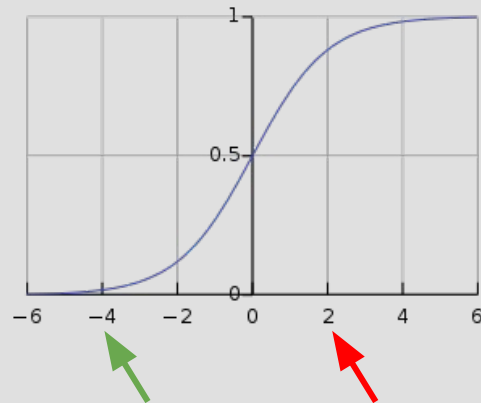
- Being correct is great but not enough
 - Great (correct prediction, far away from the decision boundary)
 - Worrisome (correct prediction, but too close to the decision boundary)
 - Bad (wrong prediction, the farther from the decision boundary the worse)



Loss

- The loss is actually a function of the distance (not the distance itself)
 - Sigmoid function:

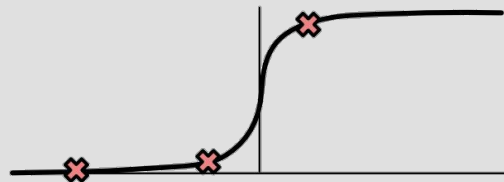
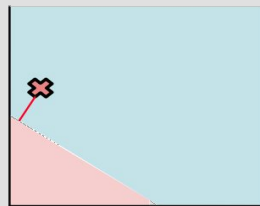
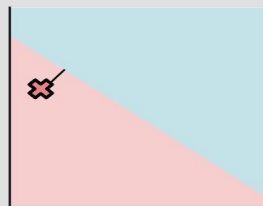
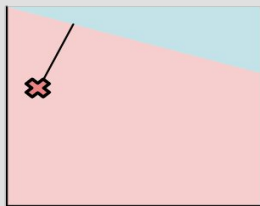
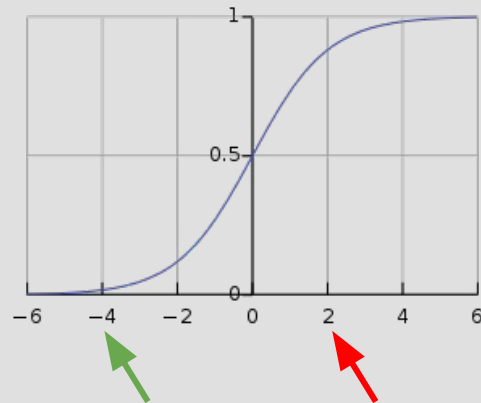
$$S(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}.$$



Loss

- The loss is actually a function of the distance (not the distance itself)
 - Sigmoid function:

$$S(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}.$$



Fitting Parameters

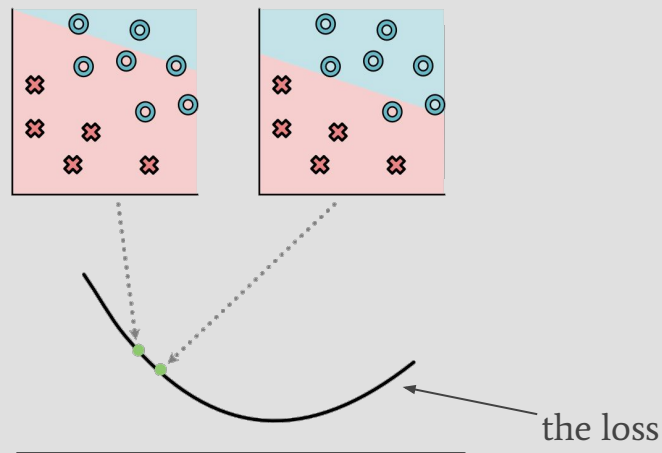
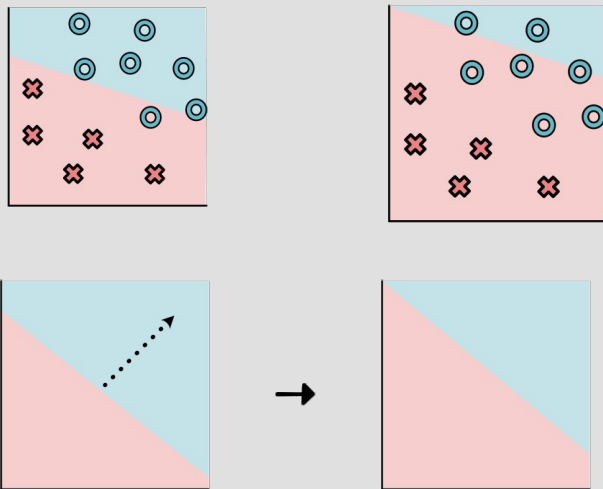
- Model class + parameters = model
 - Model class is usually fixed. The learning is limited to finding the parameters (parameter fitting)
 - Learning is guided by the loss: the lower the loss, the better the parameters (and the model)
 - The loss is usually fixed
- State-of-the-art models have billions of parameters
 - so it is complicated to find the best parameters
 - Libraries help significantly
 - Tensorflow, Pytorch, MXNet (and MiniTorch!)
- Standard recipe (gradient descent): Starting with some parameters...

We will see this in detail
(and implement it)

- 1. Compute the loss
- 2. Calculate the direction of the gradient
- 3. Change the parameters a small distance against the gradient (to make the loss smaller); go to 1

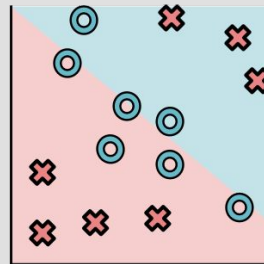
Fitting Parameters

- Standard recipe (gradient descent): Starting with some parameters...
 - 1. Compute the loss
 - 2. Calculate the direction of the gradient
 - 3. Change the parameters a small distance against the gradient (to make the loss smaller); go to 1



Neural Networks

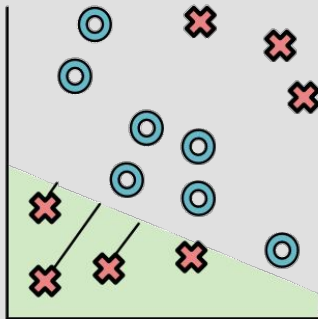
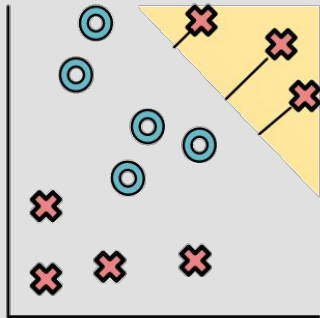
- Linear models are not enough for many datasets:



- Neural networks can specify a wider range of models
 - They divide classification in two or more stages
 - a bunch of linear classifiers to reshape the data into new points
 - a final stage (another linear classifier) over the transformed points (not the original ones!)

Neural Networks

- Neural networks can specify a wider range of models
 - They divide classification in two or more stages
 - a bunch of linear classifiers to reshape the data into new points
 - a final stage (another linear classifier) over the transformed points (not the original ones!)



One linear classifier: yellow (top right) vs. rest

One linear classifier: green (bottom left) vs. rest

Final classifier: combine them

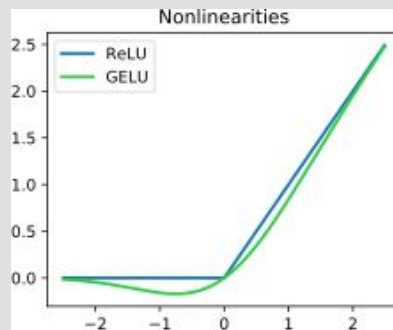
Neural Networks

- We could use the sigmoid function, but the rectified linear unit is popular

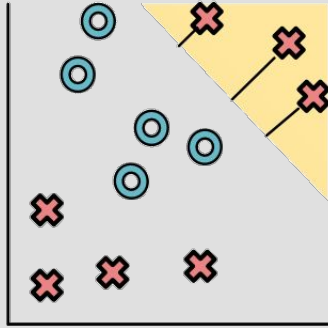
- Rectified linear unit (ReLU)

- negative becomes zero
- positive stays the same

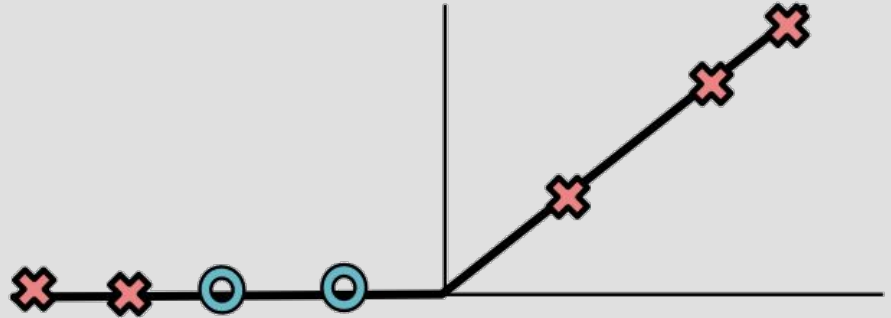
$$f(x) = x^+ = \max(0, x)$$



Neural Networks



after ReLU:



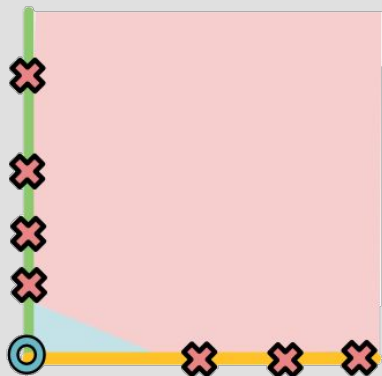
(and very similar for the green linear model)

Neural Networks

The yellow becomes x_1 (horizontal axis)

The green becomes x_2 (vertical axis)

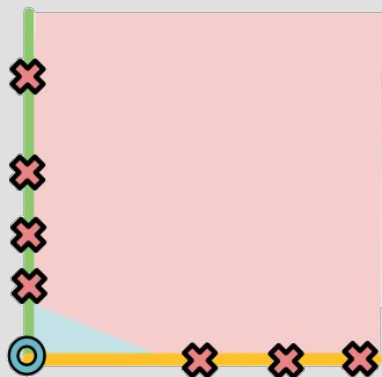
A linear classifier can now separate *the circles* and *crosses*



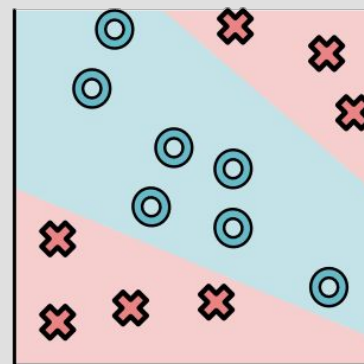
[all the circles (7) are squashed into (0, 0)]

Neural Networks

In the modified space (yellow, green coordinates)



In the original space



→ $h_1 = \text{ReLU}(x_1 \times w_1^0 + x_2 \times w_2^0 + b^0)$

→ $h_2 = \text{ReLU}(x_1 \times w_1^1 + x_2 \times w_2^1 + b^1)$

$m(x_1, x_2) = h_1 \times w_1 + h_2 \times w_2 + b$